



PROJECT DOCUMENTATION FOR HUMAN MOTION PREDICTION SYSTEM

SYSTEM DESIGN, ARCHITECTURE, PIPELINE AND IMPLEMENTATION

SUZZETH LAWANI



Table of Contents

1. Introduction	1
1.1 System Architecture	1
2. Python Project Structure	2
3. System Design.....	2
3.1. Hardware Architecture	2
3.1.1. Wiring diagram.....	2
3.1.2. Components list	3
3.1.3. Manufacturing method	4
3.2. ESP32 Firmware and Programming	5
3.3. Data collection for retraining	7
3.3.1. Raspberry Pi	7
3.3.2 Real time Pipeline	9
4. Summary of System Script Types	10
5. Mounting and Wear.....	12
6. References	13
7. Appendix.....	15
Appendix A- Gallery from the hardware build.	15

List of Figures

Figure 1- Python project structure for classification model training..... 2

Figure 2- Wiring and circuit diagram for human motion data collection hardware..... 3

Figure 3- List of components purchased for the wearable sensor hardware system..... 3

Figure 4- List of components sourced from lab/personal..... 4

Figure 5- Multiplexer bus assignments..... 6

Figure 6- Images showing the mounted sensor hardware system..... 12

1. Introduction

The purpose of this document is to highlight the technical processes taken, for both the hardware and software for the implementation of the wearable sensor hardware system.

1.1 System Architecture

In this system, real time sensor acquisition is performed by the ESP32, while the logging and the machine learning model inference is performed by the Raspberry Pi.

This project aims to achieve live classification of data; therefore, it was decided to include a Raspberry Pi with a built-in SD card. The Raspberry Pi will run the classification model. While the 32GB SD card will be able to log raw data for further analysis, improvements and long-term monitoring. The communication between the ESP32 and the Raspberry Pi will be over USB Serial. This split allows the Pi to handle the computationally heavier inference model, helping the ESP32 to save RAM.

ESP32:

Sensor → Kalman filter → preprocessing → send

Raspberry Pi:

Receive → Buffer → Windowing → Feature extraction → Feature selection → Normalisation
→ Inference → Output → Send back

ESP32:

Display the result on the OLED screen

2. Python Project Structure

```
Project
Folder: data_raw
Folder: data_processed
Folder: features
Folder: models
Python package: src
    load_data.py
    preprocess.py
    feature_engineering.py
    feature_selection.py
    normalize.py
    train.py
    evaluate.py
Python package: utils
    filters.py
    tools.py
    windowing.py
main_prelim.py
```

Figure 1- Python project structure for classification model training.

The pipeline (Figure 1) implemented in Python to train the model is coordinated by `main_prelim.py` which calls each module in sequence. Raw data is first loaded using `load_data.py`, which is then passed onto `preprocess.py` for cleaning and formatting. `feature_engineering.py` then extracts the statistical and frequency domain features from each sliding window with the windowing logic provided by `windowing.py` in the utils package. Butterworth filtering for the signal is applied through `filters.py`. The most informative features are then selected based on the parameters set within `feature_selection.py`, which are then normalised by `normalize.py` using a fitted `StandardScaler`. The processed features are then passed to `train.py` where the classification models are trained, for which `evaluate.py` generates the classification reports and confusion matrices.

3. System Design

3.1. Hardware Architecture

3.1.1. Wiring diagram

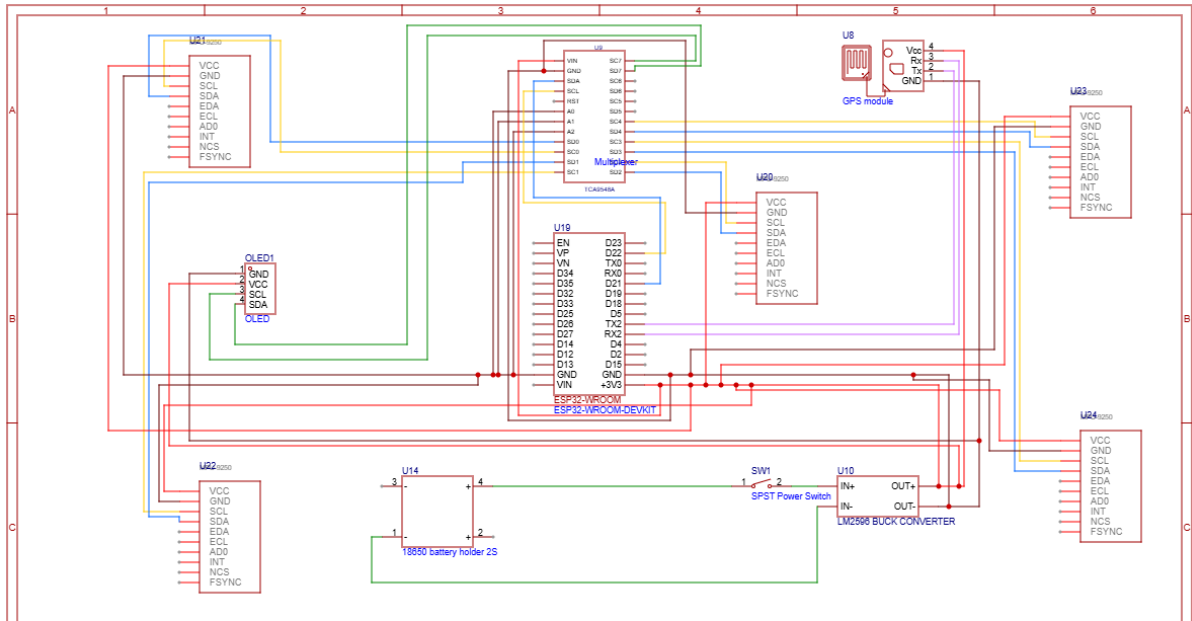


Figure 2- Wiring and circuit diagram for human motion data collection hardware.

3.1.2. Components list

Component Name	Part ID	Quantity	Supplier / Source	Unit cost (£)	Cost (£)
ESP32 WROOM-32 Development Board, Wi-Fi, Bluetooth, BLE, 2.4 GHz	Seeit ESP32-DEV-30P Bluetooth, Wi-Fi Development Board ESP32-DEV-30P	1	RS Components	14.75	14.75
MPU-9250 GY-9250	DollaTek MPU 9250 SPI/IIC 9-Axis Attitude Module Gyro Accelerator Magnetometer	5	Amazon	4.84	24.2
TCA9548A 8-channel I2C multiplexer	DollaTek TCA9548A I2C IIC Multiplexer Breakout Board 8 Channel Expansion Board	1	Amazon	3.99	3.99
Power switches	12v Awitch, Toggle Switch, Rocker Switch	1	Amazon	3.99	3.99
18650 Li-ion cell	RS PRO, 3.7V, 18650, Lithium-Ion Rechargeable Battery, 2.6Ah	2	RS Components	9.83	19.66
18650 battery holder	18650 x 2 Battery Holder 150mm Leads Hobby Electronics Electrical DIY Circuits	1	Amazon	3.95	3.95
Step Down Voltage Regulator	LM2596 DC-DC Buck Converter, Adjustable Step-Down Voltage Regulator 3-40V to 1.5-35V	1	Amazon	5.99	5.99
NEO-6M GPS module	NEO-6M GPS Flight Controller GY-NEO6MV2 Module 3V-5V	1	Amazon	5.99	5.99
128x64 OLED module (I2C)	DollaTek 2Pcs 0.96" I2C IIC SPI Serial 128x64 OLED Display Module Board	1	Amazon	5.99	5.99
PCB board	RUNCCI-YUN 48PCS PCB Board Kits, 11PCS Double Sided PCB Board Prototype Circuit Board, Printed Circuit Boards	1	Amazon	7.99	7.99
Velcro straps	1 Roll 5m Adjustable Cable Strap with 25 Buttons	1	Amazon	4.39	4.39
Chest strap	Chest Mount Harness Strap Holder	1	Amazon	12.99	12.99
					113.88

Figure 3- List of components purchased for the wearable sensor hardware system.

Component	Details
Main PCB	10CM x 22CM
Female PCB socket connectors	Assorted set
Male header pins	Assorted set
Fine insulated wire	Different colours
Connector set	Assorted pin size set
Secondary MCU	Raspberry Pi 3 model B V1.2
USB cable	USB-A to USB-C
Power bank	Capable of outputting 5V, 3A
Waist bag	

Figure 4- List of components sourced from lab/personal.

3.1.3. Manufacturing method

Images from the building process can be found in Appendix A.

1. Solder ground & power rails using copper wire along the top and bottom of the main PCB.(The main PCB is 10 x 22 CM, holding the ESP32, multiplexer, GPS, chest sensor, OLED screen and buck converter.)
2. Attach male header pins to the various components.
3. Solder female PCB socket connectors for all the components into the main PCB. Including the ESP32, multiplexer, GPS, OLED, chest sensor, buck converter.
4. Attach and solder insulated wire along the back(the side with solder contact pads) of the main PCB to the ground and power rails for each component.
5. Complete component to component wiring on the front side of the main PCB. As seen in the wiring diagram.
6. Solder male header pins to the 4 other MPU9250 sensors.
7. Solder 2 sensors to the 3x7CM PCBs for the arms and 2 sensors to 5x7CM PCBs for the legs.
8. Solder 4x receptable female connector parts onto main PCB and 4x of these onto each sensor PCB.
9. Crimp 4 strips of ribbon cables, cut according to the desired size and attach 2 male plug connectors to each side of each ribbon cables. (8 in total)
10. Complete the power circuit. Solder the ground wire of the battery holder to the IN – of the buck converter. Solder the positive wire from the battery holder to one terminal of the switch and the other switch terminal to the IN + on the buck converter.
11. Perform continuity testing using a multimeter, for the various completed connections.
12. Join all the sensor PCBs to the main PCB using the connector ribbon cables.
13. Mount the GPS antenna to the main PCB using an adhesive (double sided tape)
14. Connect the Raspberry Pi to the ESP32 using a USB A-to-C cable.
15. Attach sensor PCBs to fabrics for arms and legs and to Velcro straps (sewn).
16. Attach main PCB to chest harness (sewn).

The wiring to the connector housings for the sensor PCBs:

Top Left sensor

Sensor	Wire	Main PCB
VCC	Yellow	Power Rail
GND	Orange	GND rail
SCL	Red	SC0
SDA	Brown	SD0

Top Right sensor

Sensor	Wire	Main PCB
VCC	Green	Power Rail
GND	Blue	GND rail
SCL	Purple	SC4
SDA	Grey	SD4

Bottom Left sensor

Sensor	Wire	Main PCB
VCC	Yellow	Power Rail
GND	Orange	GND rail
SCL	Red	SC1
SDA	Brown	SD1

Bottom Right sensor

Sensor	Wire	Main PCB
VCC	Grey	Power Rail
GND	Purple	GND rail
SCL	Blue	SC5
SDA	Green	SD5

(Note: Left and Right stated above for when looking at the main PCB, from in front/above.)

3.2. ESP32 Firmware and Programming

The ESP32 was programmed using C++ in the Arduino IDE.

The libraries used include Wire, MPU9250, Adafruit_SSD1306, Adafruit_GFX, and TinyGPSPlus. (All publicly available libraries)

The ESP32 was programmed using C++ in the Arduino IDE (Integrated Development Environment), which required various libraries to be able to program the components used. Wire.h is used to communicate with devices using I2C. MPU9250 handles communication with the MPU9250 sensors. Adafruit_SSD1306 allows communication with OLED displays, while Adafruit_GFX handles the graphics for the OLED displays. Lastly, TinyGPS+ parses the NMEA sentences that the GPS outputs into usable values. (tavotech.com, 2022)

The hardware being programmed includes the ESP32, TCA9548A, 5× MPU9250, SSD1306 OLED and the NEO-6M GPS. The communication protocol to the Raspberry Pi is USB Serial, 115200 baud. The code uploaded to the ESP32 should read the raw sensor data from the body worn sensors 50 times per second, perform Kalman filtering for noise reduction and preprocess the data. The data is streamed continuously to the Raspberry Pi over USB.

Additionally, when a classification result has been determined by the Pi, the result is sent back to the ESP32 to be displayed on the OLED screen.

The multiplexer enables multiple of the IMU sensors with the same I2C address to communicate with the Microcontroller, by being on different channels. As having two devices with the same address on the same bus would cause clashing and the data to be corrupted. Additionally, the OLED screen is also connected to its own channel on the multiplexer. The TCA9548A() function needs to be called every time it is desired to switch between sensors. The multiplexer connects to the ESP32 via SDA and SCL. Whilst the GPS communicates directly with the MCU over UART protocol.

Bus	Sensor	Placement
2	MPU9250	CHEST
4	MPU9250	LEFT ARM
0	MPU9250	RIGHT ARM
5	MPU9250	LEFT LEG
1	MPU9250	RIGHT LEG
7	SSD1306	OLED SCREEN

Figure 5- Multiplexer bus assignments.

A single ESP32 firmware was used for both the data collection for retraining and deployment phases.

Upon startup the ESP32 runs the setup where the following takes place;

1. Start USB serial at 115200 baud channel to Pi (opens communication)
2. Start UART2 at 9600 baud channel to GPS (opens communication)
3. Start I2C at 400kHz communication (I2C fast mode)
4. Initialise OLED screen
5. Initialise all 5 sensors in a loop (calibrates each sensor)
6. Send column header to Pi (Pi knows what each column means)
7. Show "Ready" on OLED (signalling the system is ready)

The calibration measure the sensor's natural offset. For each sensor, the ESP32 switches to its multiplexer channel, initialises it at the I2C address 0x68, before running the calibration routines. The MPU9250 library used in the Arduino IDE has built in calibration, to remove the bias. This is suitable for development and testing. After calibration every reading is automatically corrected, which happens upon every boot and takes around 5 seconds per sensor.

Post development, another calibration method should be utilised. Calibrate all 5 MPU9250s, print the offsets to the serial monitor and copy the values. Then remove the calibration calls within the code and instead hardcode the values. Reflash the code onto the ESP32 and the boot time will be reduced significantly, with the calibration applied automatically.

Each sensor has its own Kalman filter, 50 filters in total (10 axes per each sensor). The simple 1D recursive Kalman filter smooths the noise in the sensor data. Q is the process noise; it is tuned for how fast the true signal changes and R is the measurement noise. The gyroscope is generally noisier and drifts so the Q and R values will be slightly higher.

The main loop of the code does three main things each cycle. Firstly, updating the GPS, the GPS only produces a new position once per second (1Hz). So, the position value will be repeated between updates. Secondly, at 50Hz, the ESP32 read all 5 sensors once per tick. Using a drift corrected approach, adding 20ms to the previous tick time meaning any small delay does not cause the next tick to be late, locking the timing. For each sensor, the multiplexer switches to that sensor's channel, reading all the 9 raw values and computing the acceleration magnitude. The result is sent as a comma separated line containing 50 IMU values and 6 GPS values, totalling 56 floats per line. Lastly, the ESP32 receives an activity label from the Pi, which will then update the OLED screen to display the new activity during live deployment.

3.3. Data collection for retraining

Transitioning from the preliminary study where the data was processed offline, the hardware system changes three fundamental things; the sensors, data format and the type of data processing- now being in real time. Due to these changes the model must be retrained on data collected from the hardware system.

This will be done by wearing all five sensors and performing each target activity while the ESP32 streams data to a laptop for logging. The logging script saves the CSV stream to a file including the activity label. Each session producing one CSV file of 56 columns at 50Hz. Multiple sessions of each activity are recommended to capture the natural variation in how each movement can be performed.

3.3.1. Raspberry Pi

The raspberry Pi receives the pre-processed data from the ESP32 and then applies windowing to the signals, performs feature extraction and selection, before running the classification model and yielding a result for the prediction.

This data is then used to retrain the full pipeline on the laptop before deploying the final model to the Pi.

The following steps were followed for the retraining and live deployment. (www.raspberrypi.com, n.d.) The TERMINUS application was used on a mobile phone during the data retraining for ease of data collection. (termius.com, n.d.)

[The model was trained using sklearn version 1.3.2]

1. Flash firmware onto the ESP32 and then wear the sensor hardware.
2. SSH (Secure Shell) into Raspberry Pi.
 - Find the Pi's IP address, note that the Pi and the laptop or phone has to be on the same network.
 - In the windows' cmd portal:

```
ssh pi@<ip_address>
```

- (insert the Pi's Ip address)
3. Type in Raspberry Pi password.

```
4. nano collect_data.py
```

- This allows the file to be viewed and edited directly on the Pi, if any changes are required.

5. Install the library needed:

```
pip3 install pyserial
```

6. Plug the Pi into the ESP32 via USB and then type:

```
ls /dev/tty* to confirm the USB communication serial port is present.  
(/dev/ttyUSB 0) .
```

7. Run data collection for each activity:

```
python collect_data.py walking 60
```

- Repeat for each activity, multiple sessions each, varying the time.

8. Copy raw data from the data_hardware folder on the Pi to the desired destination on the laptop. SCP (Secure Copy Protocol)

```
scp -r pi@<pi_ip_address>:/home/pi/<foldername>  
<local_destination_path>
```

9. Extract features from the raw recordings in the desired Python IDE.

```
python extract_features.py
```

10. Train the model within the Python IDE.

```
python main.py
```

11. Copy the trained models' folder to the Pi, including the trained classifier, selected features, and the scaler.

```
scp -r models/ pi@<pi_ip_address>:/home/pi/
```

12. Install the dependencies on the Pi

- SSH in and run:

```
pip3 install pyserial numpy scipy scikit-learn pandas  
joblib
```

13. Run inference on Pi:

```
python3 pi_realtime.py
```

Steps 1-7 are for data collection, 8-10 are done on the laptop, and 11-13 and for the live deployment.

3.3.2 Real time Pipeline

Once the model is trained and saved it is transferred to the Pi where a Python script runs continuously.

Stage 1- Receive and buffer

The Pi reads the USB serial connection using PySerial. Each line is one sample, 56 comma-separated floats arriving 50 times per second. The valid lines are added to a rolling buffer in memory (Liechti, n.d.).

Stage 2- Sliding window

Next a sliding window is applied so that the model classifies a window of samples instead of individually. A 5 second window, yielding 250 samples at 50Hz, gives the model sufficient signal to be able to distinguish between activities. The window slides forward by 62 samples at a time with a 75% overlap. The overlap ensures that the transitions between the activities are captured quickly instead of waiting for a completely new window to fill.

Stage 3- Feature Extraction

Statistical and frequency-based features are computed from each axis of each sensor across the window. Which condenses 250 rows x 50 columns of raw data into a single flat vector of 625 numbers (5 sensors x 10 axes x 12 features per axis)

The features computed per axis include:

Time domain- mean, standard deviation, minimum, maximum, range, skewness, kurtosis. These capture the overall magnitude and distribution shape of the signal over the window. Zero crossing rate, showing how frequently the signal crosses its mean. Jerk quantifying the rate of change of motion and sensitivity to impact events.

Frequency domain- dominant frequency and spectral energy, computed using the Fast Fourier Transform (FFT), capturing rhythmic patterns. (NumPy Developers, 2025)

The features computed per IMU: include the Signal Magnitude Area, providing a scalar measure of the overall dynamic activity, the inter axis correlations, capturing postural orientation, and the vertical dominance ratio, quantifying the vertical to lateral acceleration variability.

Stage 4- Feature selection

Including irrelevant features can reduce accuracy by introducing noise into the model input. During training, mutual information analysis identified the 150 most informative features. The name of these 20 selected features are saved to a JSON file during training and loaded by the Pi at startup. Only these 150 features out of 625 are passed to the model.

Stage 5- Normalisation

The selected 150 features operate on very different numerical scales. In this way, if the features are fed directly into the model, larger value features would dominate the model causing smaller valued ones to be ignored in comparison.

StandardScaler normalisation adjusts every feature to have a mean of zero and a standard deviation of one, allowing the features to have a more equal contribution. The scaler is fitted on the training data once and saved. At inference time on the Pi only transform() is called

and not fit() as the Pi must apply the exact same scaling parameters that were used during training. (scikit-learn, 2019)

Stage 6- Inference and Prediction Stabilisation

The normalised 150-feature vector is then passed to the loaded AdaBoost model which returns a predicted activity label. However, the raw model output may have brief transitions, unusual movement, or random noise which could lead to incorrect predictions even briefly. To address this a majority vote stabilisation is applied, where the last 6 predictions are stored in a rolling window. A label will only be confirmed, and the result be sent to the ESP32 if at least 5 of those 6 predictions agree. When a confirmed label differs from the previously confirmed label, a message is sent to update the ESP32. This method keeps the OLED display more stable and reliable.

In summary, the Raspberry Pi receives one CSV line from the ESP32, which then runs it through a pre-trained classifier, before returning it back as a single string label.

USB Serial in → Parse 56-column CSV row → Append to rolling buffer → Every 62 samples: slice last 250 rows (windowing) → Extract 625 features (time + frequency domain) → Select 150 features (from saved JSON list) → Normalise (using saved scaler) → Model inference-raw label → Majority vote- if confirmed, send label over USB Serial → ESP32 displays the label on the OLED screen.

4. Summary of System Script Types

1. Training scripts

`collect_data.py`

- Run on the Pi and records raw sensor stream to CSV.

`extract_features.py`

- Run on laptop and turns the raw CSV into feature CSV.

`main.py`

- Loads hardware CSV data, windows it, extracts features, selects features, normalises, trains AdaBoost, saves three files:

models/activity_model.pk1 → trained AdaBoost model

models/scaler.pk1 → fitted StandardScaler

models/selected_features.json → list of 150 selected feature names

2. ESP32 Firmware to be flashed onto the ESP32.

`main.ino_esp32`

- Reads sensors, Kalman filters, streams CSV to Pi, receives label back, shows on OLED.
- Runs continuously on the ESP32.

3. PI inference script to run on the Raspberry Pi.

`pi_realtime.py`

- Reads ESP32 stream, buffers, windows, extracts features, selects, normalises, runs
- AdaBoost, majority vote prediction stabilisation, sends label back. Runs continuously on the Pi.

5. Mounting and Wear

The Power bank sits in a waist mounted bag powering the Raspberry Pi, seated in the front pocket of the bag, with a USB A to C cable connecting the SBC to the ESP32 MCU. All the leg and arm sensors are mounted using Velcro straps, which are adjustable to various limb sizes. Each sensor PCB is labelled with the correct limb it was wired for, with one ribbon colour set chosen for the left side sensors (brown, red, orange, yellow) and another ribbon colour set chosen for the right side (green, blue, purple, grey). The use of ribbon cables connecting to the body mounted sensors, allowed for ease of wear and removal, while the use of sockets, instead of soldering the components directly onto the main PCB allowed for components to be easily removed if any issues did arise. The left arm, right arm, left leg and right leg sensors were mounted on outer side of the limb. While the thigh sensors were mounted approximately halfway between the hip and the knee. The chest sensor sat on the main PCB, so no further adjustments were required.

The fully mounted hardware is displayed in Figure 6.

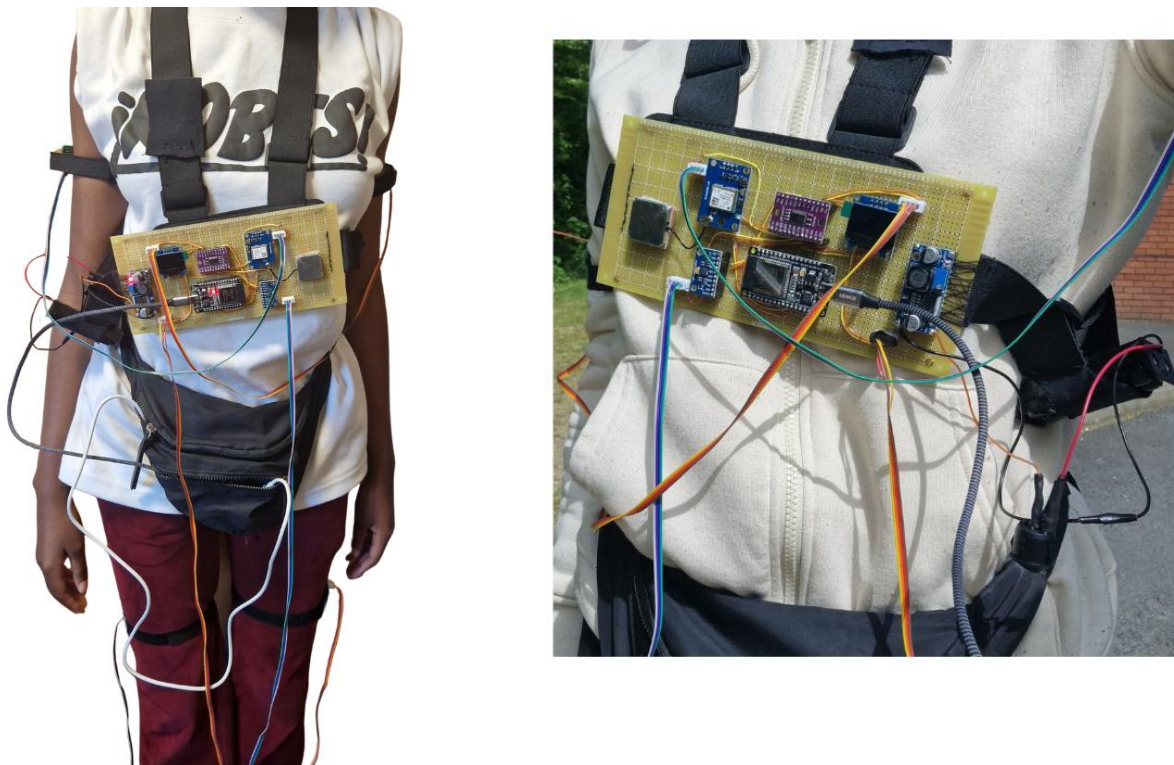


Figure 6- Images showing the mounted sensor hardware system.

6. References

38-3D (n.d.) How to Use the TCA9548A I2C Multiplexer with Arduino. Available at: <https://38-3d.co.uk/blogs/blog/how-to-use-the-tca9548a-i2c-multiplexer-with-arduino> [Accessed: 18 March 2026].

Ajay Raj (2020) Reading Multiple MPU 9250 Sensors using TCA9548A Multiplexer. Arduino Stack Exchange. Available at: <https://arduino.stackexchange.com/questions/79820/reading-multiple-mpu-9250-sensors-using-tca9548a-multiplexer> [Accessed: 12 March 2026].

Franklin, W. (2020) Kalman Filter Explained Simply. The Kalman Filter. Available at: <https://thekalmanfilter.com/kalman-filter-explained-simply/> [Accessed: 31 March 2026].

GeeksforGeeks (2025) Kalman Filter in Python. Available at: <https://www.geeksforgeeks.org/python/kalman-filter-in-python/> [Accessed: 31 March 2026].

How to Use MPU-9250/6500/9255: Pinouts, Specs, and Examples (n.d.) Cirkuit Designer. Available at: <https://docs.cirkuitdesigner.com/component/c14ee915-9533-4928-8866-55d588115daf/mpu-925065009255> [Accessed: 18 March 2026].

Liechti, C. (n.d.) pySerial — pySerial 3.5 documentation. Available at: <https://pyserial.readthedocs.io/en/latest/pyserial.html>

Magdy, K. (n.d.) ESP32 I2C: Set Pins, Multiple Devices, I2C Scanner (Arduino IDE). DeepBlueEmbedded. Available at: <https://deepblueembedded.com/esp32-i2c-tutorial-change-pins-i2c-scanner-arduino/> [Accessed: 12 March 2026].

NumPy Developers (2025) *Discrete Fourier Transform* — *NumPy v2.4 Manual*. Available at: <https://numpy.org/doc/stable/reference/routines.fft.html>

Saeed, M., Sarmad, A. & Ahmed, M.A. (2023) Wearable biomechanical monitor. GitHub repository. Available at: <https://github.com/muntadher12345/wearable-biomech-monitor> [Accessed: 29 October 2025].

Santos, R. & Santos, S. (2019) ESP32 I2C Communication: Set Pins, Multiple Bus Interfaces and Peripherals (Arduino IDE). Random Nerd Tutorials. Available at: <https://randomnerdtutorials.com/esp32-i2c-communication-arduino-ide/> [Accessed: 12 March 2026].

Santos, R. & Santos, S. (n.d.) Guide for TCA9548A I2C Multiplexer: ESP32, ESP8266, Arduino. Random Nerd Tutorials. Available at: <https://randomnerdtutorials.com/tca9548a-i2c-multiplexer-esp32-esp8266-arduino/> [Accessed: 12 March 2026].

scikit-learn Developers (2026) StandardScaler — scikit-learn 1.8.0 documentation. Available at: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>

Sensirion Software (2024) *Using a Multiplexer to Connect Multiple Sensors*. Hackster.io. Available at: <https://www.hackster.io/sensirion-software/using-a-multiplexer-to-connect-multiple-sensors-1363d0> [Accessed: 18 March 2026].

TavoTech (2026) GPS: GGA, RMC, and GLL NMEA sentence structure. Available at: <https://tavotech.com/gps-nmea-sentence-structure/> [Accessed: 30 April 2026].

termius.com. (n.d.). *Termius - SSH platform for Mobile and Desktop*. [online] Available at: <https://termius.com/>.

www.raspberrypi.com. (n.d.). *Raspberry Pi Documentation - Remote Access*. [online]
Available at: <https://www.raspberrypi.com/documentation/computers/remote-access.html>.

7. Appendix

Appendix A- Gallery from the hardware build.

